

借助 Jev 构建运行框架

Building a Harness with Jev

LangChain · Sydney Runkle, Hunter Lovell · 2026-09-18 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

Agent 在循环中运行：LLM 决定做什么，工具执行操作，模型评估结果，然后继续循环，直到任务完成。

起初，Agent 和 LLM 很难集成到软件应用中，因为软件应用依赖结构化数据与可预测的接口。后来出现了两项基础能力，让集成容易了许多：

- **工具调用**让模型能够发出结构化请求，并接收结构化结果。
- **结构化输出**让模型能够返回按结构组织的结果。

但即使具备了这些能力，Agent 循环仍然**缓慢而且昂贵**：每一次决策，都需要再调用一次模型。

这时，Jev 出现了。Jev 是 TypeSafe AI 发布的一个新模型。该公司报告称，在分类任务上，相比同类 LLM，Jev 的**推理速度最高可达 200 倍，成本可低至 1/400**。

本文将介绍 Jev 如何工作、它适合放在 Agent 循环的哪个位置，以及如何与 LangChain 一起使用。

了解 Jev

实际上，Jev 不是传统意义上的 LLM，它不会生成文本。TypeSafe AI 团队将其称为 System One 模型：

📖 System One 模型是一类 AI 模型，专门用于快速做出结构化决策，供软件直接使用。System One 模型评估一个**状态**，并返回带类型的答案及概率。

它使用面向校准决策的强化学习（RLCD）进行训练。你的代码可以利用这些结果，指导 Agent 接下来做什么，而不必为每次决策都进行一次完整的聊天 LLM 调用。

调用 Jev 模型时，你需要向它发送一个**状态**（即上下文），以及有关该状态的**问题**。下面是其文档中客服工单示例的单问题版本：

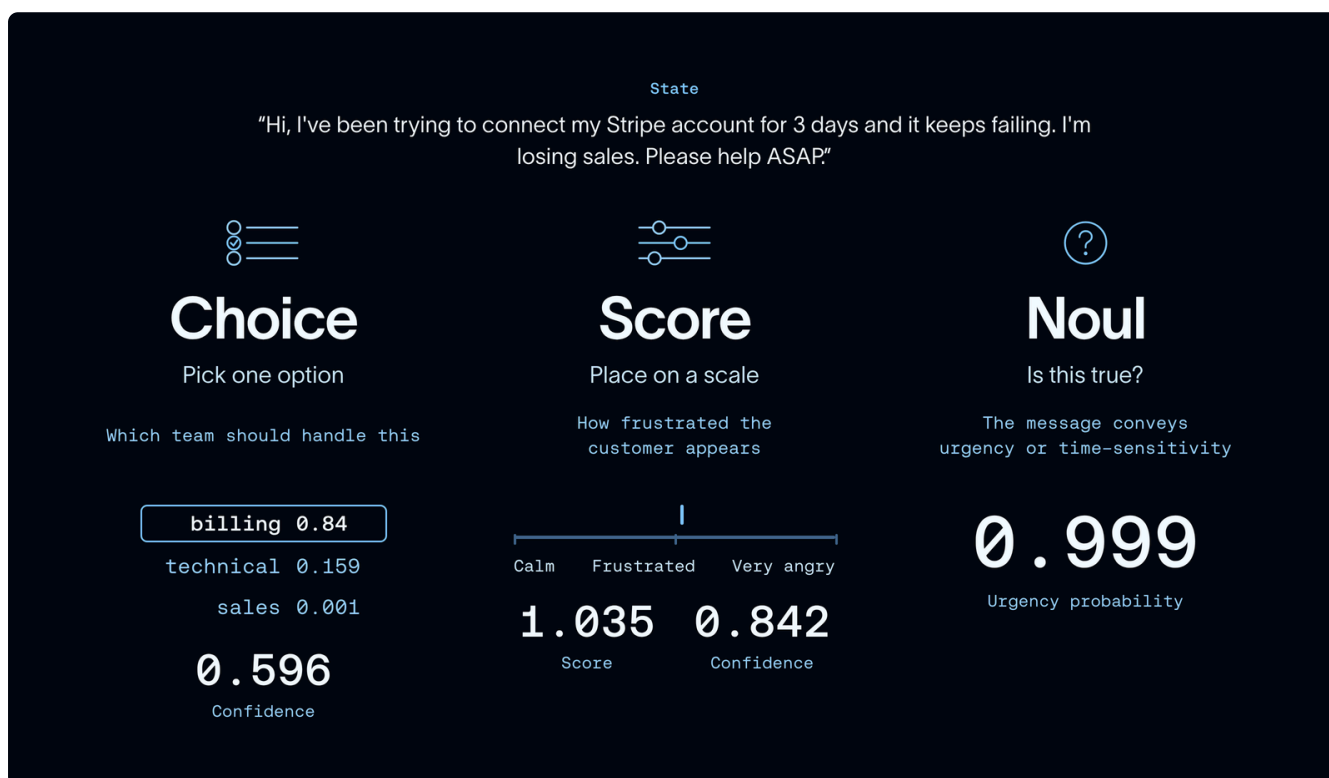
```
{
  "model": "jev-latest",
  "state": "Hi, I've been trying to connect my Stripe account for 3 days and it keeps failing. I'm losing sales. Please help ASAP.",
  "questions": {
    "is_urgent": {
      "type": "noul",
      "instructions": "The message conveys urgency or time-sensitivity"
    }
  }
}
```

文档示例给出了下面这个关于紧急程度的答案。这里省略了响应中的其他部分：

```
{
  "is_urgent": {
    "type": "noul",
    "noul": 0.999
  }
}
```

这表示该消息有 99.9% 的概率属于紧急事项，你的应用可以据此优先处理该工单。

它支持三种问题类型：



- **Choice:** 从一组选项中选择。返回每个选项的概率，以及一个总体置信度分数。
- **Score:** 按照低、中、高等有序等级对输入评分。返回一个连续分数、其底层分布，以及一个置信度值。
- **Noul:** 回答是非问题。返回某个陈述为真的概率。

这里有一个关键特性：你可以在一次请求中，针对同一个状态提出多个问题。

💡 System One 模型会并行评估请求中的所有问题。增加问题几乎不会改变响应时间，只会增加额外问题所消耗的 token 成本，而这部分成本很低。

如需查看针对一张客服工单提出多个问题的示例，请参阅 [TypeSafe 快速入门](#)。

总之，与传统 LLM 不同，Jev 既不受文本生成过程的限制，也不受顺序决策的限制！

如何结合 LangChain 使用 Jev

LangChain 不依赖特定供应商的设计，非常适合在支持数千种其他集成和模型供应商的同时，支持 Jev。

LangChain 集成通过 `TypeSafeClassifier` 提供 Jev。你将状态与问题传给 `.invoke()`，得到的会是分类结果，而不是聊天响应。

安装 `langchain-typesafe`，设置 `TYPESAFE_API_KEY`，然后发起调用：

```
from langchain_typesafe import Noul, TypeSafeClassifier

classifier = TypeSafeClassifier()

response = classifier.invoke({
    "state": (
        "The deploy failed twice and customers are seeing 500s. "
        "Can someone look now?"
    ),
    "questions": {
        "urgent": Noul(
            instructions="Does this need attention right now?"
        ),
    },
})

urgency = response.nouls["urgent"].noul
```

状态可以是文本、结构化数据，或者 LangChain 消息。因此，你可以在节点或中间件钩子中，直接利用 Agent 已有的上下文调用 Jev。

你可以将这种能力加入自定义中间件或工具中！

使用场景

Jev 并不能直接替代 LLM。它不会生成文本，但能以更低的时延与成本，处理我们如今经常交给 LLM 的分类任务。这使它有望成为驱动 Agent 的模型的有益补充：用 LLM 进行开放式推理和生成，用 Jev 在过程中快速做出结构化决策。

模型路由

简单查询与困难的调试任务，并不需要使用同一个模型。模型路由中间件让 Jev 评估请求，并根据你定义的标准选择模型：简单任务使用快速、便宜的模型，复杂任务则使用能力更强的模型。

```
from langchain.agents import create_agent
from langchain_typesafe.experimental.middleware import (
    ModelChoice,
    ModelRouterMiddleware,
)

router = ModelRouterMiddleware(
    choices={
        "fast": ModelChoice(
            model="openai:luna",
            criteria="Direct lookups, extraction, and localized changes.",
        ),
        "powerful": ModelChoice(
            model="openai:sol",
            criteria="Architecture and high-stakes decisions.",
        ),
    },
    instructions="Choose the least costly model that can complete the task.",
)

agent = create_agent("openai:gpt-5.6-luna", middleware=[router])
```

路由器根据最新的用户消息选择模型，并在整个运行过程中使用它。相应的概率与置信度也会保留在 Agent 状态中，供后续使用。

自动模式

Agent 本质上仍然不值得完全信任。Agent 可能接收到不当指令，无论这些指令是自然出现的，还是由足够有动机的攻击者构造的；这些指令都可能诱使它采取我们不希望发生的行动。

Claude、Codex、Cursor 等编程运行框架，已经提供了某种机制，在危险操作执行之前对其进行分类。这逐渐帮助人们建立起对 Agent 的信任。但直到现在，这个分类步骤一直封装在运行框架的闭源部分中。

如今，既便宜又高效的分类模型已经出现，我们就可以采用同样的模式，并将其应用到所有 Agent 上！

```
from langchain.agents import create_agent
from langchain_typesafe.experimental.middleware import (
    AutoModeMiddleware,
)

guardrail = AutoModeMiddleware(tools=["bash"])

agent = create_agent("openai:gpt-5.6-luna", middleware=[guardrail])
```

AutoModeMiddleware 使用 Jev 检查工具调用中可能包含的高风险决策，并在工具执行前阻止这些调用。

开始使用！

我们对 Jev 及其带来的可能性十分兴奋。我们已经看到了一些很有意思的项目：Browserbase 的 Kyle Jeong 正在以不到一美分的成本驱动浏览器操作 Agent；Jarrod Watts 构建了一个实时交易 Agent；Ryan Vogel 则在大规模进行邮件分类处理。

如今几乎每周都有新模型发布，但这个模型引起的反响格外强烈。我们很期待看到你用 LangChain 和 Jev 构建出什么。

欢迎在论坛上分享你的想法，在 X 上提及我们并展示你正在构建的东西，或者参与 LangChain issues 的讨论！